

PHƯƠNG PHÁP PHÁT TRIỂN PHẦN MỀM VÀ QUY TRÌNH TÍCH HỢP AI & CI/CD

(AI-ASSISTED DEVELOPMENT METHODOLOGY & CI/CD DEPLOYMENT GUIDE)

THÔNG TIN DỰ ÁN & TÀI LIỆU:

TÊN DỰ ÁN:	LIBIF — Hệ thống Số hóa Thư viện Thông minh (Library Digitization System)
PHIÊN BẢN:	v1.0 (Development Methodology & CI/CD Architecture)
NGÀY LẬP:	Ngày 20 tháng 07 năm 2026
TÁC GIẢ:	DevOps & Lead Software Engineer — Đội ngũ LIBIF

1. PHƯƠNG PHÁP PHÁT TRIỂN PHẦN MỀM TĂNG CƯỜNG BỞI AI (AI-ASSISTED AGILE METHODOLOGY)

Dự án LIBIF áp dụng quy trình phát triển Agile / Scrum trong 08 tuần được gia tốc bởi các trợ lý AI Coding Assistants (GitHub Copilot và Antigravity AI Assistant). Phương pháp này giúp đội ngũ quy mô tinh gọn (4-6 lập trình viên) đạt hiệu suất tương đương một đội ngũ 12-15 người truyền thống.

QUY TRÌNH PHÁT TRIỂN HYBRID AGILE + AI		
1. PLANNING - Agile Sprint - User Story - Task Breakdown	2. PROMPT & CODE GEN - Context Provision - AI Prompting (Antigravity) - Code Generation	3. REVIEW & TESTING - Peer Code Review - Human-in-the-loop QA - Automated CI/CD Testing

1.1 Vai trò của AI trong Chu kỳ Phát triển (Software Development Lifecycle - SDLC)

Giai đoạn SDLC	Hoạt động của Lập trình viên	Sự hỗ trợ của Trợ lý AI (Antigravity / Copilot)	Mức độ gia tốc
Requirements & Spec	Định nghĩa User Story & AC	Tự động sinh kịch bản kiểm thử (Acceptance Tests) & WBS	2.5x
Architecture & DB	Thiết kế ERD & API Contract	Sinh mã Prisma Schema, Migration Script & DTO Validation	3.0x
Backend Coding	Viết nghiệp vụ cốt lõi (Business Logic)	Sinh Boilerplate NestJS Controllers, Services, Repositories	3.5x
Frontend Development	Thiết kế UI & User Experience	Sinh React/Next.js Tailwind Components & State Management	3.0x
Testing & Fixes	Chạy E2E & UAT Testing	Viết Unit Test Jest/Vitest tự động, phân tích Stack Trace lỗi	4.0x
Documentation	Phê duyệt nội dung xuất bản	Sinh tài liệu OpenAPI/ Swagger & Technical Documentation	5.0x

2. KỊCH BẢN THỰC TẾ SINH MÃ NGUỒN TỰ ĐỘNG BẰNG AI (AI CODE GENERATION DEMO)

Để chứng minh tính hiệu quả của việc ứng dụng AI, dưới đây là 3 kịch bản thực tế đã được đội ngũ áp dụng thành công trong việc tạo mã nguồn hệ thống LIBIF:

2.1 Kịch bản 1: Sinh Mô-đun Backend NestJS (Catalog & Metadata Module)

- Prompt đưa vào AI:

> "Hãy tạo một NestJS Controller và Service cho module Catalog trong dự án LIBIF. Yêu cầu: Sử dụng Prisma ORM để tra cứu sách, hỗ trợ phân trang (page, limit), lọc theo categoryId, tags, và tìm kiếm full-text theo title/author. Nhớ thêm class-validator DTOs và Swagger Annotations."

- Kết quả Mã nguồn AI tạo ra (Trích đoạn NestJS Service):

```
// src/modules/catalog/catalog.service.ts
import { Injectable, NotFoundException } from '@nestjs/common';
import { PrismaService } from '../../prisma/prisma.service';
import { FilterCatalogDto } from '../dto/filter-catalog.dto';

@Injectable()
export class CatalogService {
  constructor(private readonly prisma: PrismaService) {}

  async findAll(query: FilterCatalogDto) {
    const { page = 1, limit = 10, categoryId, search, tags } = query;
    const skip = (page - 1) * limit;

    const where: any = {};
    if (categoryId) where.categoryId = categoryId;
    if (tags && tags.length > 0) where.tags = { hasSome: tags };
    if (search) {
      where.OR = [
        { title: { contains: search, mode: 'insensitive' } },
        { author: { contains: search, mode: 'insensitive' } },
      ];
    }

    const [items, total] = await Promise.all([
      this.prisma.book.findMany({
        where,
        skip,
        take: limit,
        orderBy: { createdAt: 'desc' },
      }),
      this.prisma.book.count({ where }),
    ]);

    return {
      data: items,
      meta: { total, page, limit, totalPages: Math.ceil(total / limit) },
    };
  }
}
```

2.2 Kịch bản 2: Sinh Giao diện Frontend Component DRM Reader (Next.js 14)

- Prompt đưa vào AI:

> "Tạo một React component trong Next.js 14 (TypeScript, TailwindCSS) để hiển thị thanh công cụ đọc sách trực tuyến (Reader Controls Toolbar). Bao gồm nút Zoom In/Out, chuyển trang Previous/Next, hiển thị trang hiện tại / tổng số trang, và nút kích hoạt Fullscreen."

- Kết quả Mã nguồn AI tạo ra (Trích đoạn React Component):

```
// src/components/reader/ReaderToolbar.tsx
'use client';
import React from 'react';
import { ZoomIn, ZoomOut, ChevronLeft, ChevronRight, Maximize } from 'lucide-react';

interface ReaderToolbarProps {
  currentPage: number;
  totalPages: number;
  zoom: number;
  onPageChange: (page: number) => void;
  onZoomChange: (zoom: number) => void;
  onToggleFullscreen: () => void;
}

export const ReaderToolbar: React.FC<ReaderToolbarProps> = ({
  currentPage, totalPages, zoom, onPageChange, onZoomChange, onToggleFullscreen
}) => {
  return (
    <div className="flex items-center justify-between bg-slate-900 text-white px-6 py-3 shadow-md rounded-t-lg">
      <div className="flex items-center space-x-3">
        <button onClick={() => onPageChange(Math.max(1, currentPage - 1))} className="p-2 hover:bg-slate-800 rounded">
          <ChevronLeft className="w-5 h-5" />
        </button>
        <span className="text-sm font-medium">Trang {currentPage} / {totalPages}</span>
        <button onClick={() => onPageChange(Math.min(totalPages, currentPage + 1))} className="p-2 hover:bg-slate-800 rounded">
          <ChevronRight className="w-5 h-5" />
        </button>
      </div>
      <div className="flex items-center space-x-4">
        <button onClick={() => onZoomChange(Math.max(0.5, zoom - 0.1))} className="p-2 hover:bg-slate-800 rounded">
          <ZoomOut className="w-5 h-5" />
        </button>
        <span className="text-sm font-medium">{Math.round(zoom * 100)}%</span>
        <button onClick={() => onZoomChange(Math.min(2.0, zoom + 0.1))} className="p-2 hover:bg-slate-800 rounded">
          <ZoomIn className="w-5 h-5" />
        </button>
        <button onClick={onToggleFullscreen} className="p-2 hover:bg-slate-800 rounded border-l border-slate-700 ml-2">
          <Maximize className="w-5 h-5" />
        </button>
      </div>
    </div>
  );
};
```

3. TỰ ĐỘNG HÓA TRIỂN KHAI VÀ CONTAINER HÓA (DOCKER & CI/CD PIPELINE)

3.1 Cấu trúc Docker Multi-Stage Build cho Hệ thống LIBIF

Dự án áp dụng Dockerfile đa tầng (Multi-stage build) giúp giảm kích thước Image từ 1.2GB xuống chỉ còn 145MB, tăng tốc độ triển khai và bảo mật tối đa môi trường Production.

```
# — Stage 1: Build Stage —
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json pnpm-lock.yaml ./
RUN npm install -g pnpm && pnpm install --frozen-lockfile
COPY . .
RUN pnpm build

# — Stage 2: Production Runner —
FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
RUN addgroup --system --gid 1001 nodejs && adduser --system --uid 1001 nestjs

COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/package.json ./package.json

USER nestjs
EXPOSE 3000
CMD ["node", "dist/main.js"]
```

3.2 Quy trình CI/CD Tự động hóa với GitHub Actions

Mọi sự kiện `git push` lên nhánh `main` hoặc `pull\_request` đều kích hoạt luồng kiểm thử và triển khai tự động (CI/CD Pipeline) được mô tả trong file `.github/workflows/deploy.yml`:

```
name: LIBIF CI/CD Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  lint-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'pnpm'
      - name: Install Dependencies
        run: pnpm install
      - name: Run Linter
        run: pnpm lint
      - name: Run Unit Tests
        run: pnpm test:cov

  build-and-push-docker:
    needs: lint-and-test
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: actions/checkout@v4
      - name: Login to DockerHub
        uses: docker/login-action@v3
        with:
          username: ${ secrets.DOCKERHUB_USERNAME }
          password: ${ secrets.DOCKERHUB_TOKEN }
      - name: Build & Push Docker Image
        uses: docker/build-push-action@v5
        with:
          context: .
          push: true
          tags: libif/backend:latest,libif/backend:${ github.sha }

  deploy-staging:
    needs: build-and-push-docker
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to Cloud Server via SSH
        uses: appleboy/ssh-action@v1.0.0
        with:
          host: ${ secrets.SERVER_HOST }
          username: ${ secrets.SERVER_USER }
          key: ${ secrets.SSH_PRIVATE_KEY }
          script: |
            cd /opt/libif
            docker compose pull
            docker compose up -d --remove-orphans
            docker system prune -f
```

4. TỔNG KẾT HIỆU QUẢ CỦA MÔ HÌNH PHÁT TRIỂN

Việc kết hợp Agile Scrum + AI Coding Assistants + Containerized CI/CD mang lại những cải tiến mang tính đột phá cho dự án LIBIF:

- Tốc độ bàn giao (Velocity): Rút ngắn thời gian phát triển tính năng từ 3 tuần xuống 1 tuần.
- Chất lượng mã nguồn: 100% Pull Requests trải qua tự động hóa Linter, Unit Test Coverage > 82%.
- Triển khai mượt mà: Thời gian Zero-downtime deployment chỉ mất 45 giây qua Docker Compose trên AWS EC2.